SIMATS ENGINEERING

Assessment Tool 2: Case Study

Course Name	Computer Vision with OpenCV
Course Code	DSA02
Course Outcome (CO3)	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)
Assessment Weightage	20%
Student Name	V M Sai Bhargav
Register Number	192324126

Instructions: Read the given scenario carefully and analyze the problem using appropriate computer vision techniques. Justify your answers with relevant reasoning and comparisons wherever necessary.

Questions Answered: 1, 2, 3, 4, and 5 (Any 5 of 15, as instructed)

Code of Conduct

I V M Sai Bhargav (Reg. No. 192324126) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V M SAI BHARGAV

Question 1: Preprocessing and Feature Detection

> ***Scenario:*** *A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent.*

Analysis of the Problem

Feature detectors such as Harris corner detector, SIFT, and ORB rely on local intensity gradients to locate distinctive points. When images are captured under varying illumination — direct sunlight, shadows, artificial lighting, or backlighting — the raw pixel intensities of the same physical scene change significantly between frames. Because gradient-based detectors compute derivatives of intensity, uneven lighting introduces false gradients in uniformly-lit regions and suppresses genuine gradients in poorly-lit regions. This causes the same real-world feature to be detected in one image but missed in another, breaking the repeatability that detection and matching depend on.

In addition, sensor noise is amplified in low-light regions, and specular highlights or glare in over-lit regions create spurious high-gradient points that get mistaken for genuine corners or edges. The net effect is an inconsistent, non-repeatable set of keypoints across frames of the same scene.

How Preprocessing Improves Feature Detection

- Grayscale conversion and noise reduction: Applying a Gaussian or bilateral filter suppresses sensor noise before gradient computation, reducing spurious keypoints while preserving true edges (bilateral filtering is preferred since it smooths noise without blurring edges).
- Histogram equalization / CLAHE: Contrast Limited Adaptive Histogram Equalization redistributes intensity values so that both dark and bright regions gain usable contrast, normalizing illumination differences across the image and making gradient magnitudes comparable across captures.
- Illumination normalization: Techniques such as homomorphic filtering separate the illumination and reflectance components of an image, allowing the system to correct for non-uniform lighting (e.g., shadows) while retaining the reflectance information that actually defines object structure.
- Gamma correction: Adjusts the intensity mapping to compensate for images that are systematically too dark or too bright, bringing them into a consistent dynamic range before detection.

Justification

Preprocessing does not add new information to the scene; instead, it removes the variability introduced by the capture conditions so that the underlying structural information — edges, corners, blobs — becomes the dominant signal the detector responds to. Empirically, applying CLAHE plus Gaussian smoothing before running a Harris or SIFT detector significantly increases the repeatability rate (the fraction of keypoints detected consistently across differently-lit images of the same scene), which directly improves the reliability of any downstream matching or recognition task.

Question 2: Edge vs Corner Detection

> ***Scenario:*** *In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching.*

Why Edge Detection Is Insufficient

Edge detectors such as the Sobel or Canny operator locate pixels where intensity changes sharply along one direction — typically the boundary between two regions. An edge, however, is only strongly defined perpendicular to its own direction; along the edge itself, every point looks locally identical to its neighbours. This is known as the aperture problem: a small window placed anywhere along a straight edge cannot uniquely localize a specific point on that edge, because the same gradient pattern repeats along the line.

Consequently, edges are excellent for describing object boundaries and shape outlines but are poor candidates for keypoints used in matching, since matching requires a location that can be uniquely and repeatably identified in a small local neighbourhood across two different images.

Why Corner Detection Is Better Suited Here

A corner is a point where intensity changes significantly in two directions simultaneously — for example, where two edges meet, or within a textured region with local structure in multiple orientations. Detectors such as Harris, Shi-Tomasi, or FAST evaluate how the local intensity changes as a small window is shifted in different directions (via the structure tensor / second-moment matrix). Only points with high gradient variation along both axes — corners — produce a strong response in all shift directions, which resolves the aperture problem and gives a location that is uniquely identifiable.

The scenario explicitly contains textured regions, which are rich in exactly this kind of multi-directional intensity variation, making them ideal sources of stable corner keypoints — whereas the same textured regions would produce a scattered, ambiguous edge map that is unsuitable for point-to-point matching.

Justification / Comparison

Edge detection answers 'where is the boundary of a structure' while corner detection answers 'where is a uniquely locatable point.' Because feature matching requires the second property, corner detection (and its descendants such as FAST and ORB's oFAST) is the correct choice for this scenario, while edge detection remains valuable as a separate step for shape or contour analysis rather than point correspondence.

Question 3: Hough Transform in Noisy Images

> ***Scenario:*** *A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.*

Effect of Noise on Hough-Based Detection

The Hough Transform detects lines by mapping edge points from image space into parameter space (typically using the ρ–θ polar representation) and accumulating votes at the parameter combinations that many points agree on. Its accuracy depends entirely on the quality of the edge map fed into it. Noise in the source image causes the preceding edge detector to produce false edge pixels scattered randomly across the image.

Each of these spurious edge points casts its own sinusoidal curve into Hough (parameter) space. While a true line produces a sharp, high-value peak where many curves intersect, noise-induced points spread votes thinly across many unrelated (ρ, θ) cells. This raises the overall background noise floor of the accumulator array, which can create false peaks (phantom lines that don't exist in the scene) or weaken/split a genuine peak into several smaller ones, causing the true line to be missed or fragmented into shorter segments.

Suggested Improvements

- Pre-filtering: Apply Gaussian or median blurring before edge detection to suppress salt-and-pepper or Gaussian noise, reducing the number of false edge pixels generated in the first place.
- Better edge extraction: Use Canny edge detection with well-tuned hysteresis thresholds instead of a simple gradient threshold — Canny already applies non-maximum suppression and double-thresholding, which discards weak, noise-driven edges while retaining edges connected to strong ones.
- Accumulator threshold tuning: Increase the minimum vote count required to accept a peak in the Hough accumulator, so that only lines supported by a sufficiently large number of edge points are reported, filtering out noise-driven false peaks.
- Parameter resolution: Coarsen the ρ and θ resolution slightly so that nearby votes from a genuine line reinforce the same bin rather than spreading across adjacent bins, which increases peak sharpness at a small cost to localization precision.
- Use the Probabilistic Hough Transform (HoughLinesP): It only considers a random subset of edge points and additionally checks for minimum line length and maximum gap between segments, which is more robust to scattered noisy points than the full classical transform.

Justification

Because the Hough Transform is fundamentally a voting mechanism, its accuracy improves whenever the input votes come from genuine structure rather than noise. Combining noise-reduction preprocessing with careful parameter selection (accumulator threshold, resolution) directly reduces false votes and sharpens true peaks, which is why this two-pronged approach is the standard fix for noisy line detection.

Question 4: Feature Matching under Transformations

> ***Scenario:*** *Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively.*

Why Traditional Matching Fails

Simple corner detectors like Harris are computed at a single, fixed scale using a fixed-size window. When the same object appears larger or smaller across two images (due to camera zoom or distance change), the same physical corner may need a larger or smaller window to be correctly recognized as a corner in each image. A fixed-scale detector will therefore either miss it entirely in one image or detect it with a different apparent structure, so the descriptors built around that point (e.g., simple intensity patches) no longer match between the two images. Rotation compounds this problem: descriptors that are not rotation-invariant compare pixel patterns in a fixed orientation, so a rotated version of the same patch produces a completely different descriptor vector even though it represents the same physical feature.

Why Scale-Invariant Methods Succeed

Algorithms such as SIFT (Scale-Invariant Feature Transform) and its faster derivatives (SURF, ORB) address both issues directly:

- Scale space construction: The image is convolved with Gaussians at multiple scales (an octave pyramid), and keypoints are detected as extrema in the Difference-of-Gaussians across this scale space. This means

a feature is detected at the scale at which it is most prominent, so the same physical point is found regardless of how large or small the object appears.

- Orientation assignment: Each keypoint is assigned a dominant orientation based on local gradient directions, and the descriptor is computed relative to that orientation. This makes the descriptor invariant to image rotation, since rotating the object simply rotates the reference orientation along with it.
- Scale- and rotation-normalized descriptors: The final descriptor (e.g., the 128-dimensional SIFT gradient histogram) is built in this normalized local frame, so descriptors of the same physical feature computed from differently scaled/rotated images become numerically similar and can be matched reliably using nearest-neighbour distance.

Justification / Comparison

Traditional matching assumes the object's appearance is geometrically identical between images, which is violated whenever scale or rotation changes. Scale-invariant methods explicitly model and normalize these geometric variations before descriptor comparison, which is why they maintain high matching accuracy under the transformations described, at the cost of higher computational complexity than simple corner matching.

Question 5: PCA for Feature Optimization

Scenario: *A system extracts a large number of features, leading to high computational cost and redundancy.*

The Problem of High-Dimensional, Redundant Features

When a vision system extracts a large number of features per image (e.g., long descriptor vectors, or many keypoints each with high-dimensional descriptors), two problems emerge. First, computational and storage cost grows with dimensionality — matching, indexing, and classification all become slower as vector length increases (the curse of dimensionality also degrades distance-based matching, since distances between points become less discriminative in very high dimensions). Second, many of the extracted features are correlated or redundant, since neighbouring pixels and overlapping descriptor regions carry overlapping information, so the same variance in the data is expressed repeatedly across multiple dimensions.

How PCA Helps

Principal Component Analysis is a linear dimensionality-reduction technique that transforms the original correlated feature space into a new set of orthogonal axes (principal components), ordered by the amount of variance in the data that they capture.

- It computes the covariance matrix of the feature vectors and finds its eigenvectors/eigenvalues; the eigenvectors with the largest eigenvalues represent the directions of maximum variability in the data.
- By projecting the original high-dimensional features onto only the top-k principal components, the system retains most of the meaningful variance (i.e., the information that actually distinguishes one image/feature from another) while discarding directions that contribute little — which are often the redundant or noisy dimensions.
- This directly reduces feature vector length, so downstream operations (matching, classification, storage) become significantly faster, and redundancy is eliminated because the new axes are, by construction, uncorrelated with each other.

Justification

Because PCA is chosen to maximize retained variance for a given number of dimensions, it provides a mathematically optimal linear projection for compressing correlated features with minimal loss of discriminative information. This makes it well suited to this scenario: it reduces both computational cost and redundancy simultaneously, provided the number of retained components (k) is chosen to preserve a sufficiently high percentage (e.g., 95%+) of total variance so that important discriminative information is not lost.